AI Career Guidance System — Architecture & Schema Design

A system that combines resume/ATS scoring, skill-gap assessment, and personalized career path recommendations.

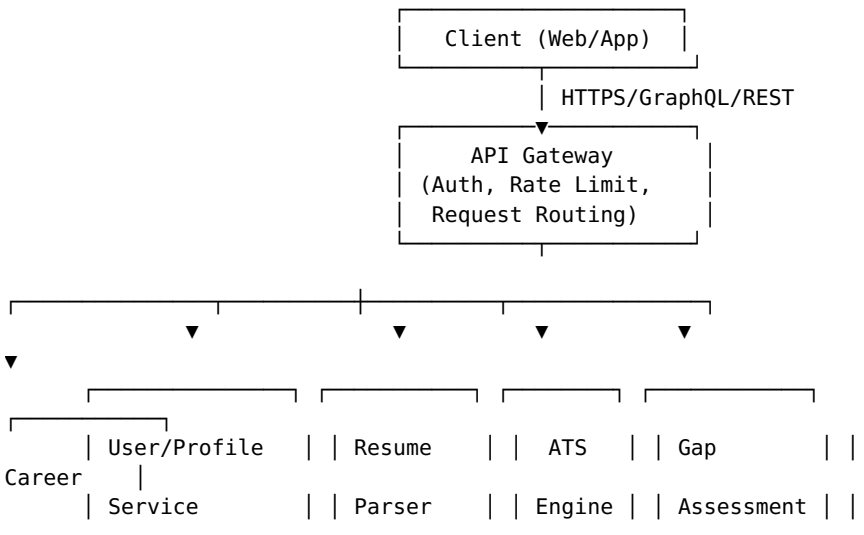
---

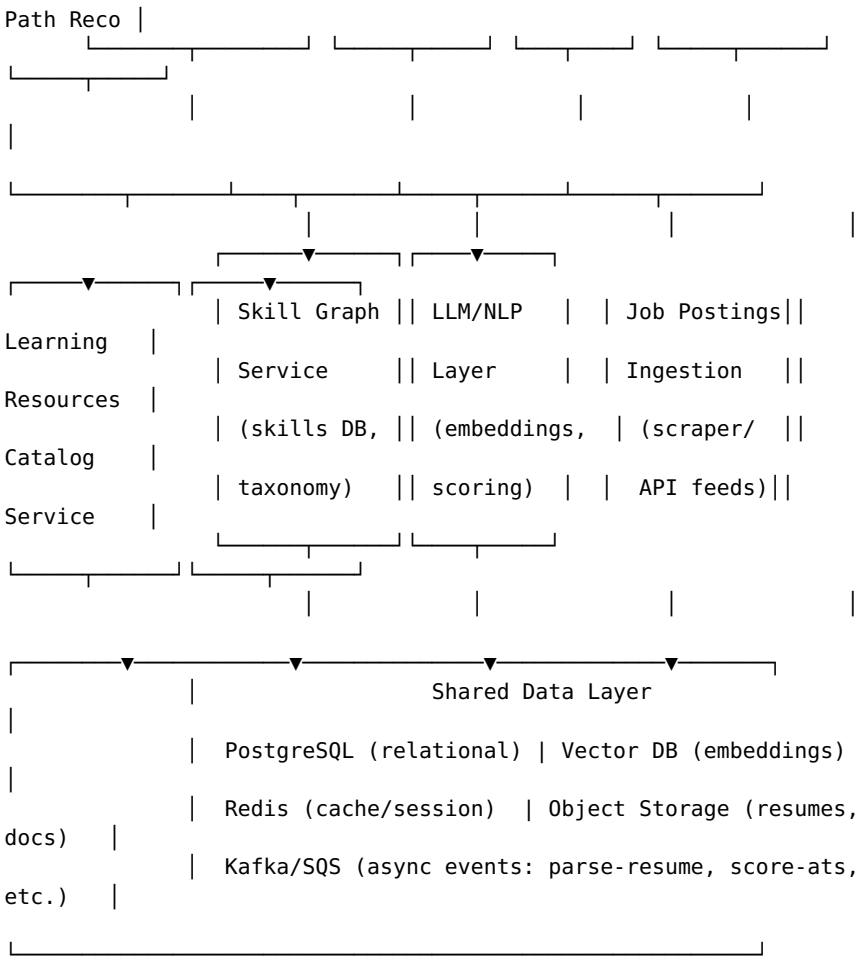
1. System Modules

Module	Responsibility
User & Profile Service	Auth, profile, education, work history, preferences
Resume Parser Service	Extract structured data from uploaded resumes (PDF/DOCX)
ATS Scoring Engine	Score resume against a job description (keyword match, formatting, structure)
Skill Extraction Service	Pull skills from resume, job descriptions, and market data using NLP/LLM
Gap Assessment Engine	Compare current skills vs. target role skills; produce gap report
Career Path Recommender	Suggest roles/paths based on profile, market trends, gap results
Learning Resource Service	Map identified gaps to courses/certifications
Job Matching Service	Match user profile to open job postings
Notification/Reporting Service	Progress tracking, alerts, PDF reports
Admin/Analytics Service	Aggregate insights, usage metrics, model feedback loop

---

2. High-Level Architecture





Key architectural choices: - **Event-driven core**: Resume upload → async parse → async ATS score → async gap analysis, decoupled via a message queue so heavy NLP/LLM calls don't block the API. - **Vector DB** (e.g. pgvector, Pinecone, Qdrant) stores skill/job/resume embeddings for semantic matching (a job requiring "Node.js" should match a resume listing "Express.js" even without exact keyword overlap). - **Skill Graph/Taxonomy** is a normalized, versioned source of truth (e.g. built on ESCO/O*NET or a custom taxonomy) so skills extracted by different services stay consistent. - **LLM layer** is abstracted behind an internal interface so you can swap providers/models for parsing, scoring rationale, and recommendation generation.

---

3. Database Schema (Relational Core)

3.1 Users & Profiles

```
        -- Core identity
        users (
          id UUID PK,
          email VARCHAR UNIQUE NOT NULL,
          password_hash VARCHAR,
          full_name VARCHAR,
          role VARCHAR CHECK (role IN ('candidate','admin','recruiter')),
          created_at TIMESTAMP,
          updated_at TIMESTAMP
        )

        profiles (
```

```
  id UUID PK,
  user_id UUID FK -> users.id,
  headline VARCHAR,
  current_title VARCHAR,
  years_experience NUMERIC,
  location VARCHAR,
  target_roles TEXT[],          -- desired career paths
  target_industries TEXT[],
  bio TEXT,
  updated_at TIMESTAMP
)

education (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  institution VARCHAR,
  degree VARCHAR,
  field_of_study VARCHAR,
  start_date DATE,
  end_date DATE
)

work_experience (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  company VARCHAR,
  title VARCHAR,
  description TEXT,
  start_date DATE,
  end_date DATE,
  is_current BOOLEAN
)
```

3.2 Skills & Taxonomy

```
skills (
  id UUID PK,
  name VARCHAR UNIQUE NOT NULL,
  category VARCHAR,              -- e.g. 'technical','soft','domain'
  taxonomy_source VARCHAR,       -- e.g. 'ESCO', 'custom'
  embedding VECTOR(1536)         -- for semantic similarity
)

skill_aliases (
  id UUID PK,
  skill_id UUID FK -> skills.id,
  alias VARCHAR                  -- e.g. 'JS' -> 'JavaScript'
)

user_skills (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  skill_id UUID FK -> skills.id,
  proficiency_level SMALLINT,    -- 1-5 or
beginner/intermediate/expert
  source VARCHAR,                --
'resume_extracted','self_reported','assessed'
  evidence_ref UUID,             -- FK to resume_id or assessment_id
  confidence_score NUMERIC,      -- model confidence in the
extraction
  last_used_date DATE
)
```

3.3 Resumes & Parsing

```
        resumes (
          id UUID PK,
          profile_id UUID FK -> profiles.id,
          file_url VARCHAR,              -- object storage path
          file_type VARCHAR,
          raw_text TEXT,
          parsed_json JSONB,             -- structured parse output
          version INT,
          is_primary BOOLEAN,
          uploaded_at TIMESTAMP
        )

        resume_sections (
          id UUID PK,
          resume_id UUID FK -> resumes.id,
          section_type VARCHAR,          --
'summary','experience','education','skills','projects'
          content TEXT,
          order_index INT
        )
```

3.4 Job Postings & Descriptions

```
        job_postings (
          id UUID PK,
          source VARCHAR,                --
'internal','linkedin_api','scraped'
          external_id VARCHAR,
          title VARCHAR,
          company VARCHAR,
          raw_description TEXT,
          parsed_requirements JSONB,      -- structured: required skills,
experience, education
          seniority_level VARCHAR,
          location VARCHAR,
          posted_at TIMESTAMP,
          embedding VECTOR(1536)
        )

        job_required_skills (
          id UUID PK,
          job_posting_id UUID FK -> job_postings.id,
          skill_id UUID FK -> skills.id,
          importance VARCHAR,             --
'required','preferred','nice_to_have'
          min_proficiency SMALLINT
        )

        career_roles (                    -- canonical role definitions, not
tied to a specific posting
          id UUID PK,
          title VARCHAR UNIQUE,
          description TEXT,
          typical_seniority VARCHAR,
          avg_salary_range JSONB,
          growth_outlook VARCHAR          -- e.g. from labor market data
        )

        career_role_skills (
          id UUID PK,
          career_role_id UUID FK -> career_roles.id,
          skill_id UUID FK -> skills.id,
          importance VARCHAR,
          min_proficiency SMALLINT
        )
```

```
career_paths (                    -- role -> role progression edges
  id UUID PK,
  from_role_id UUID FK -> career_roles.id,
  to_role_id UUID FK -> career_roles.id,
  typical_years NUMERIC,
  notes TEXT
)
```

3.5 ATS Scoring

```
ats_scans (
  id UUID PK,
  resume_id UUID FK -> resumes.id,
  job_posting_id UUID FK -> job_postings.id NULLABLE,  -- null if
generic scan
  overall_score NUMERIC,          -- 0-100
  keyword_match_score NUMERIC,
  formatting_score NUMERIC,
  structure_score NUMERIC,
  readability_score NUMERIC,
  created_at TIMESTAMP
)

ats_scan_details (
  id UUID PK,
  ats_scan_id UUID FK -> ats_scans.id,
  category VARCHAR,               --
'missing_keyword','formatting_issue','section_issue'
  detail TEXT,
  severity VARCHAR,                -- 'low','medium','high'
  suggestion TEXT
)

ats_matched_keywords (
  id UUID PK,
  ats_scan_id UUID FK -> ats_scans.id,
  keyword VARCHAR,
  matched BOOLEAN,
  frequency_in_resume INT,
  frequency_in_jd INT
)
```

3.6 Gap Assessment

```
gap_assessments (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  target_role_id UUID FK -> career_roles.id NULLABLE,
  target_job_posting_id UUID FK -> job_postings.id NULLABLE,
  overall_readiness_score NUMERIC, -- 0-100
  created_at TIMESTAMP
)

gap_assessment_items (
  id UUID PK,
  gap_assessment_id UUID FK -> gap_assessments.id,
  skill_id UUID FK -> skills.id,
  required_level SMALLINT,
  current_level SMALLINT,
  gap_size SMALLINT,               -- required - current
  priority VARCHAR                  -- 'critical','moderate','minor'
)

learning_resources (
```

```
  id UUID PK,
  title VARCHAR,
  provider VARCHAR,
  url VARCHAR,
  resource_type VARCHAR,           -- 'course','certification','article','project'
  duration_hours NUMERIC,
  cost NUMERIC,
  skill_ids UUID[]                 -- or normalized junction table
)

gap_recommendations (
  id UUID PK,
  gap_assessment_item_id UUID FK -> gap_assessment_items.id,
  learning_resource_id UUID FK -> learning_resources.id,
  rank INT
)
```

3.7 Recommendations & Tracking

```
career_recommendations (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  career_role_id UUID FK -> career_roles.id,
  match_score NUMERIC,
  rationale TEXT,                  -- LLM-generated explanation
  generated_at TIMESTAMP
)

progress_tracking (
  id UUID PK,
  profile_id UUID FK -> profiles.id,
  gap_assessment_item_id UUID FK -> gap_assessment_items.id,
  status VARCHAR,                   -- 'not_started','in_progress','completed'
  updated_at TIMESTAMP
)
```

---

4. Key Workflows

A. Resume Upload → ATS Score 1. Client uploads resume → stored in object storage, `resumes` row created. 2. Event `resume.uploaded` → Resume Parser Service extracts text/sections → `resume_sections`, `parsed_json` populated. 3. Skill Extraction Service runs NLP/LLM over parsed text → populates `user_skills` with `source='resume_extracted'`. 4. If a target job/role is specified, ATS Engine compares parsed resume to `job_required_skills` / raw JD text → keyword, formatting, structure scoring → `ats_scans` + `ats_scan_details`.

B. Gap Assessment 1. User selects a target role or pastes a target JD. 2. System pulls `career_role_skills` (or extracts skills from JD via the same NLP pipeline). 3. Gap Engine diffs `user_skills` (current levels) vs required levels → `gap_assessment_items` with computed `gap_size` and `priority`. 4. Learning Resource Service matches each gap item to `learning_resources` via skill_id → `gap_recommendations`.

C. Career Path Recommendation 1. Recommender pulls profile embedding (skills + experience + preferences) and compares against `career_roles` embeddings in the vector DB. 2. Combines semantic similarity + market signals (growth_outlook, salary) + gap-size penalty (roles requiring huge unrealistic gaps score lower). 3. LLM generates natural-language rationale per recommendation → stored in `career_recommendations.rationale`.

5. Suggested Tech Stack

Layer	Options
API Gateway	Kong / AWS API Gateway / NGINX
Backend services	Node.js (NestJS) or Python (FastAPI) — Python is a natural fit given NLP-heavy services
Relational DB	PostgreSQL (+ pgvector for embeddings, or a separate vector DB)
Vector DB (alt.)	Qdrant, Pinecone, Weaviate
Cache/session	Redis
Async messaging	Kafka or AWS SQS/SNS
Object storage	S3 / GCS for resumes and generated reports
NLP/LLM layer	Claude (or other LLM) for parsing, skill extraction, rationale generation; spaCy for lightweight NER pre-processing
Search	OpenSearch/Elasticsearch for keyword-based ATS matching
Observability	Prometheus + Grafana, OpenTelemetry tracing
Deployment	Kubernetes, with each service independently scalable (NLP services are CPU/GPU heavy, others are lightweight)

6. Design Notes & Trade-offs

- **Separating ATS scoring from Gap assessment**: they share the skill-extraction pipeline but serve different purposes — ATS is about *passing an automated filter for a specific job*, gap assessment is about *long-term skill development toward a role*. Keeping them as distinct services with a shared skill graph avoids conflating "resume optimization" with "genuine capability."
- **Confidence scores on extracted skills** matter: a skill mentioned once in a bullet point is weaker evidence than one backed by multiple experience entries or a certification. This affects both ATS matching and gap sizing.
- **Versioning resumes and assessments**: keep historical `ats_scans` and `gap_assessments` rather than overwriting, so users (and you) can track improvement over time.
- **Bias/fairness**: if job matching or scoring uses demographic-adjacent proxies (school name, employer prestige), audit for disparate impact — this is a common failure mode in ATS-like systems.

Want me to go deeper on any one piece — e.g. the ATS scoring algorithm itself, the embedding-based job-matching logic, or an ER diagram rendered visually?